

Sequenzdiagramm – StudyQuest

Titel des Dokuments:

Sequenzdiagramme – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.4

Datum:

2. März 2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbeschreibung
---------	-------	-------	-----------------------

0.1	25.10.2025	G. Carrano	Erstes Sequenzdiagramm (UC01 – Registrierung) erstellt
0.2	03.11.2025	G. Carrano	Diagramme für UC04–UC06 (Quest-Lifecycle) ergänzt
1.0	14.11.2025	M. Steer	Erstfassung der Sequenzdiagramm erstellt
1.1	21.11.2025	M. Steer	Diagramme für UC09–UC15 hinzugefügt
1.2	22.11.2025	G. Carrano	Fehler in UC05-Diagramm behoben (DB-Aufrufe korrigiert)
1.3	25.02.2026	M. Steer	Alternativszenario ergänzt, Finalversion zur Abgabe vorbereitet

1.4	02.03.2026	M. Steer	Szenariobeschreibung Übersichtstabelle und Requirement-Verfahren ergänzt
-----	------------	-------------	--

Distribution List

Name	Rolle	
Sascha Wanninger	Prüfer / Betreuer	
Michael Steer	Scrum Master	
Luke Engehardt	Product Owner	
Giuliana Carrano	Developer	

Paul Strasser	Developer	
Roman Faber	Developer	

Übersicht

Die folgenden Sequenzdiagramme beschreiben die dynamischen Abläufe aller 15 Use Cases des StudyQuest-Systems. Jedes Diagramm zeigt die Interaktion zwischen den beteiligten Akteuren, UI-Komponenten, Anwendungslogik und der Datenschicht. Die Nummerierung (SQ01–SQ15) korrespondiert direkt mit den Use Cases (UC01–UC15) aus der Anforderungsanalyse.

ID	Use Case	Beteiligte Klassen
SQ01	UC01 – Registrieren & Einloggen	User, UI, DB
SQ02	UC02 – Passwort zurücksetzen	User, UI, DB
SQ03	UC03 – Profil & Einstellungen verwalten	User, UI, DB

SQ04	UC04 – Lernquest starten	User, Quest, UI, DB
SQ05	UC05 – Lernquest abschließen (XP & Level-Up)	User, Quest, LearningSession, GameRule, Achievement, Notification
SQ06	UC06 – Lern-Session per Timer tracken	User, LearningSession, Quest, UI
SQ07	UC07 – Noten & Module verwalten	User, Grade, UI, DB
SQ08	UC08 – Fortschritt & Dashboard einsehen	User, Quest, Grade, Achievement, Leaderboard, UI
SQ09	UC09 – Benachrichtigungen & Reminder verwalten	User, Notification, UI

SQ10	UC10 – Noten importieren / exportieren	User, Grade, UI
SQ11	UC11 – Achievements / Badges freischalten	User, Achievement, Notification
SQ12	UC12 – Leaderboard & Streaks anzeigen	User, Leaderboard, UI
SQ13	UC13 – Quest-Katalog verwalten (Admin)	Admin, Quest, GameRule, UI, DB
SQ14	UC14 – XP- und Levelregeln konfigurieren (Admin)	Admin, GameRule, UI, DB
SQ15	UC15 – Benutzerkonten administrieren (Admin)	Admin, User, UI, DB

SQ01 – Registrieren & Einloggen (UC01)

Akteure: Studierende:r

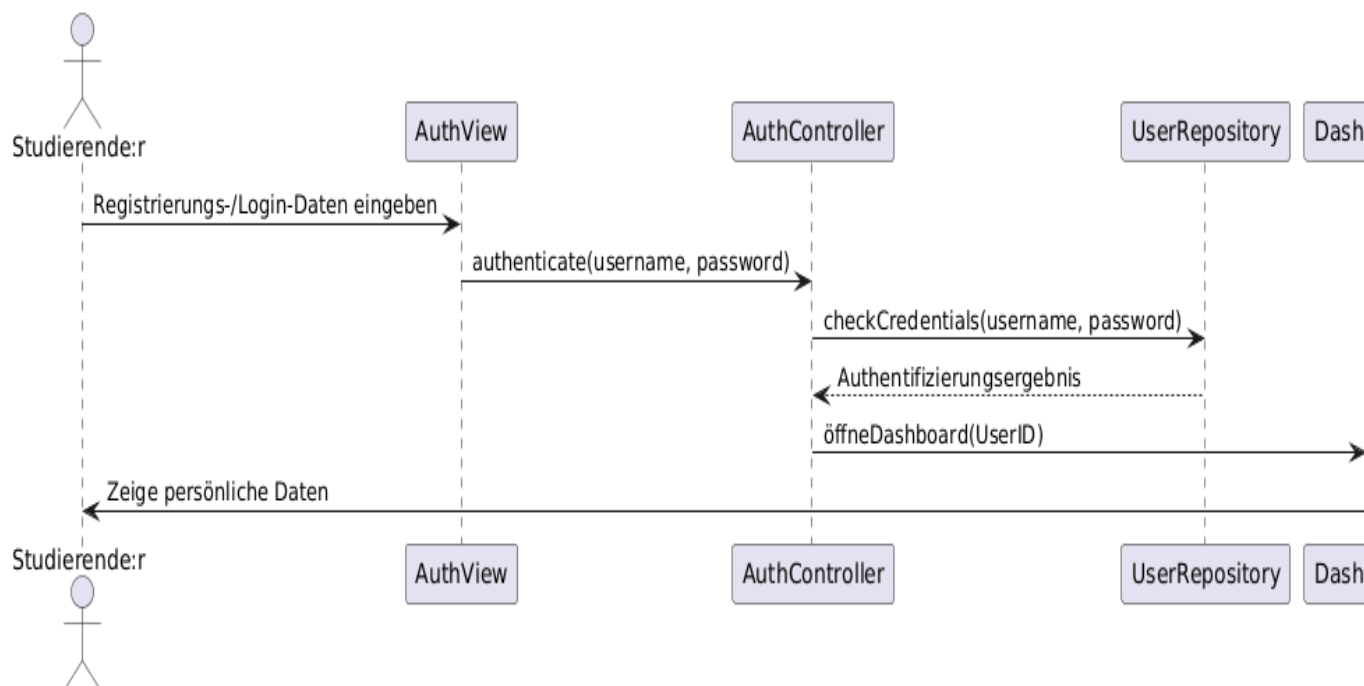
Beteiligte Klassen: User, UI (AuthView), DB

Referenzierte Requirements: UC01-F1, UC01-F2, UC01-F3, UC01-F4, UC01-NF1, UC01-NF2

Szenario A: Registrierung

Hauptablauf: 1. Studierende:r öffnet die Registrierungsseite im Browser. 2. UI zeigt das Registrierungsformular (E-Mail, Passwort, Name) an. 3. Studierende:r gibt Registrierungsdaten ein und klickt „Registrieren“. 4. UI sendet die Eingabedaten an die User-Klasse zur Validierung. 5. User.register() prüft, ob die E-Mail bereits existiert (UC01-F2). 6. Bei neuer E-Mail: Passwort wird gehasht (UC01-NF1) und ein neuer Benutzer wird in der DB angelegt. 7. UI zeigt Erfolgsmeldung und leitet zum Login weiter.

Alternativpfad: - **5a:** E-Mail bereits vorhanden → Einheitliche Fehlermeldung „Registrierung fehlgeschlagen“ wird angezeigt (UC01-NF2), kein Account wird erstellt. - **4a:** Ungültige Eingaben (leere Felder, ungültiges E-Mail-Format) → UI zeigt Validierungsfehler.

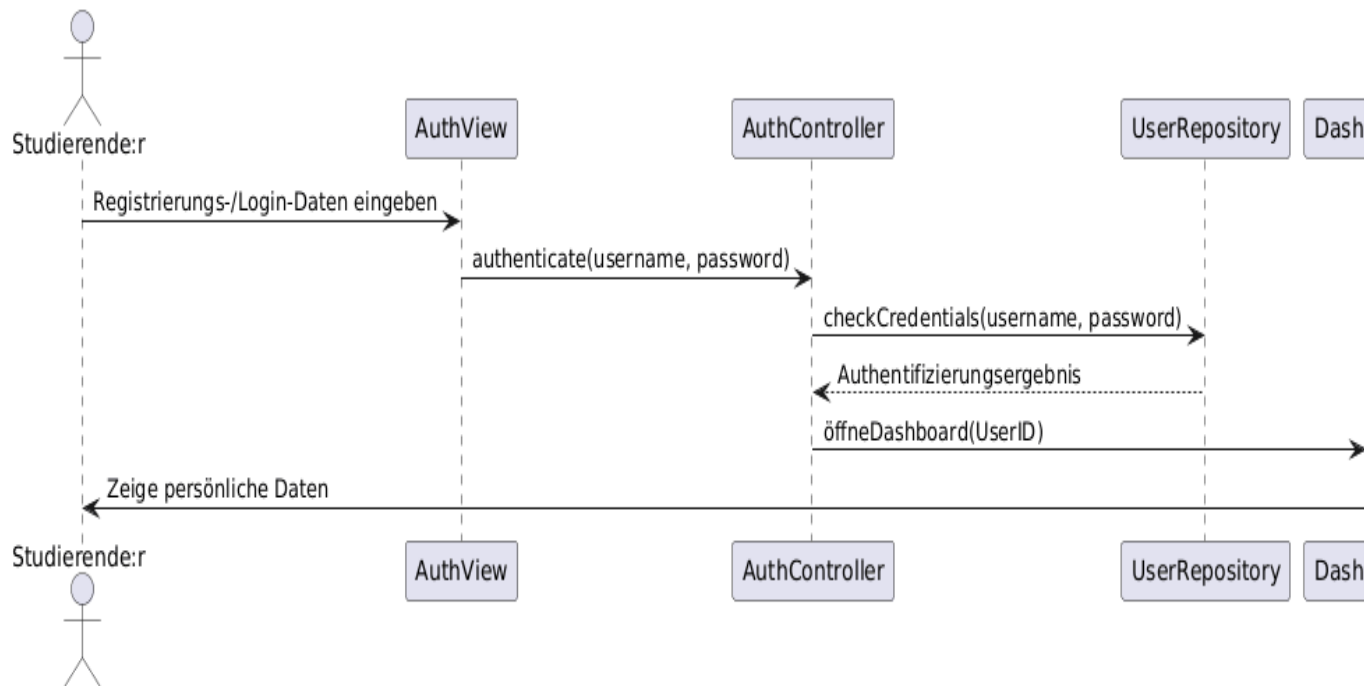


Szenario B: Login

Hauptablauf: 1. Studierende:r öffnet die Login-Seite. 2. UI zeigt Login-Formular (E-Mail, Passwort). 3. Studierende:r gibt Zugangsdaten ein und

klickt „Einloggen“. 4. UI ruft User.login() mit E-Mail und Passwort auf. 5. User.login() prüft Credentials gegen die DB (Passwort-Hash-Vergleich). 6. Bei erfolgreicher Authentifizierung: Session wird erstellt, Dashboard wird geladen (UC01-F4).

Alternativpfad: - 5a: Ungültige Zugangsdaten → Einheitliche Fehlermeldung „E-Mail oder Passwort falsch“ (UC01-NF2).



SQ02 – Passwort zurücksetzen (UC02)

Akteure: Studierende:r

Beteiligte Klassen: User, UI (AuthView), DB

Referenzierte Requirements: UC02-F1, UC02-F2, UC02-F3, UC02-F4, UC02-NF1, UC02-NF2

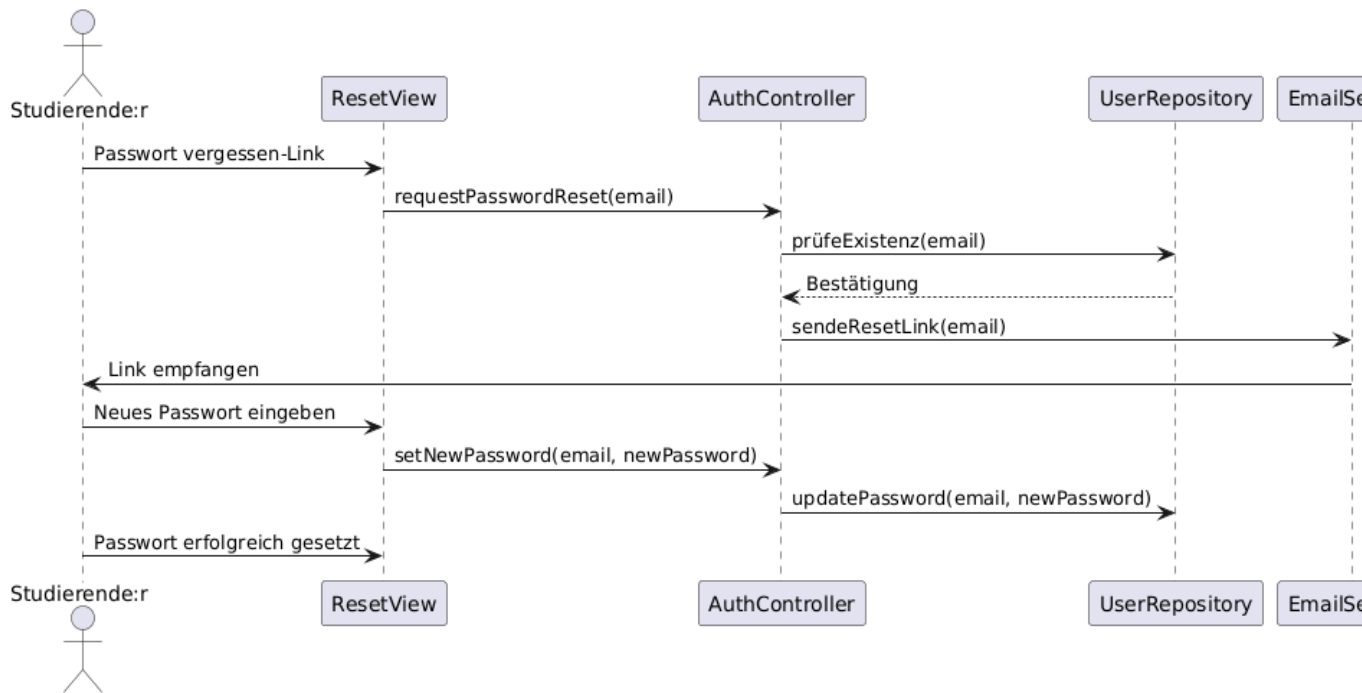
Status: Nicht implementiert (Deviation)

Hauptablauf:

1. Studierende:r klickt auf „Passwort vergessen“ auf der Login-Seite.

2. UI zeigt ein Eingabefeld für die E-Mail-Adresse.
3. Studierende:r gibt die E-Mail-Adresse ein und bestätigt.
4. System generiert einen zeitlich begrenzten Reset-Token (30 min Gültigkeit, UC02-F2).
5. System sendet E-Mail mit Reset-Link (unabhängig davon, ob die E-Mail existiert → UC02-NF1).
6. Studierende:r klickt den Link und gibt ein neues Passwort ein (UC02-F3).
7. System validiert den Token, hasht das neue Passwort und speichert es.
8. Altes Passwort wird ungültig (UC02-F4).

Alternativpfad: - **4a:** E-Mail existiert nicht → Gleiche neutrale Antwort „E-Mail gesendet, falls registriert“ (UC02-NF1). - **6a:** Token abgelaufen → Fehlermeldung „Link ist abgelaufen“ (UC02-NF2).



SQ03 – Profil & Einstellungen verwalten (UC03)

Akteure: Studierende:r

Beteiligte Klassen: User, UI (ProfileView), DB

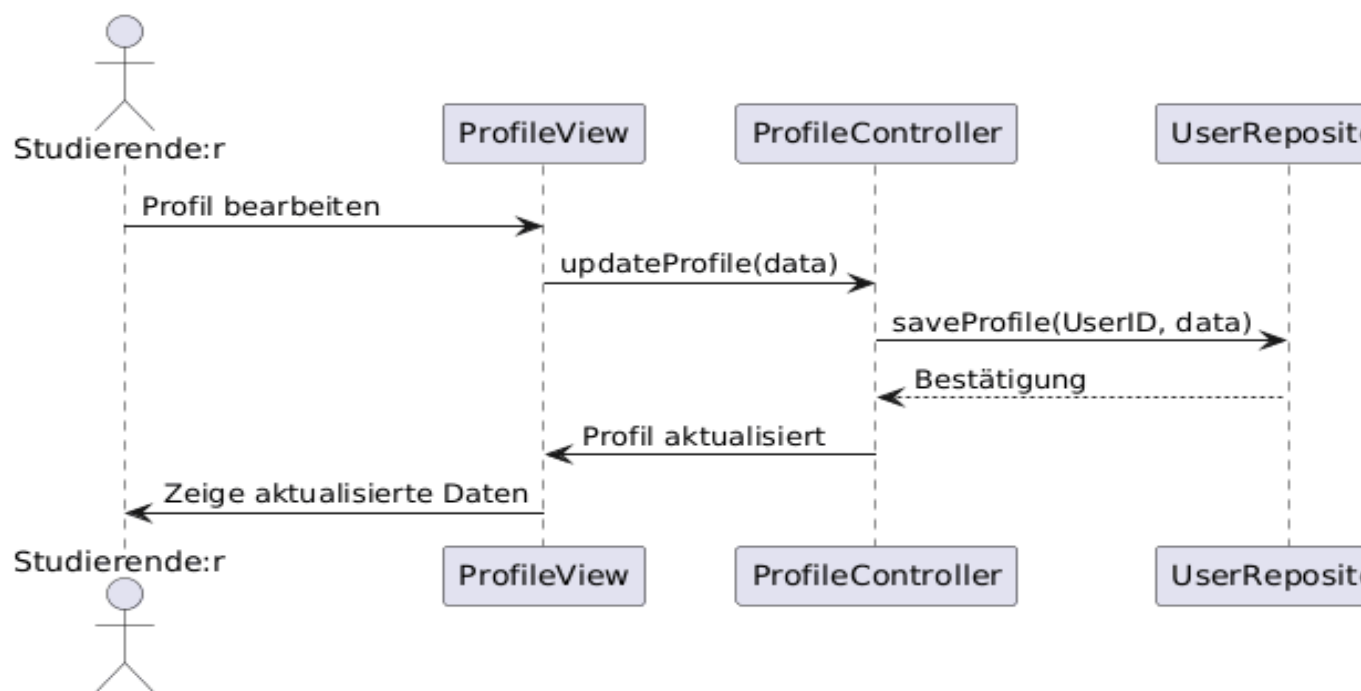
Referenzierte Requirements: UC03-F1, UC03-F2, UC03-F3, UC03-NF1, UC03-NF2

Hauptablauf:

1. Studierende:r navigiert zur Profilseite (Authentifizierung wird geprüft, UC03-NF1).
2. UI ruft User-Daten aus der DB ab und zeigt aktuelle Profildaten an (UC03-F1).
3. Studierende:r ändert Name, Avatar oder Studiengang (UC03-F2).

4. UI sendet die geänderten Daten an User.updateProfile().
5. User.updateProfile() validiert die Eingaben und speichert in der DB.
6. UI bestätigt die erfolgreiche Speicherung, aktualisierte Daten werden angezeigt (UC03-F3).

Alternativpfad: - **1a:** Nicht authentifiziert → Redirect zum Login. - **5a:** Ungültige Eingaben → Verständliche Fehlermeldung (UC03-NF2).



SQ04 – Lernquest starten (UC04)

Akteure: Studierende:r

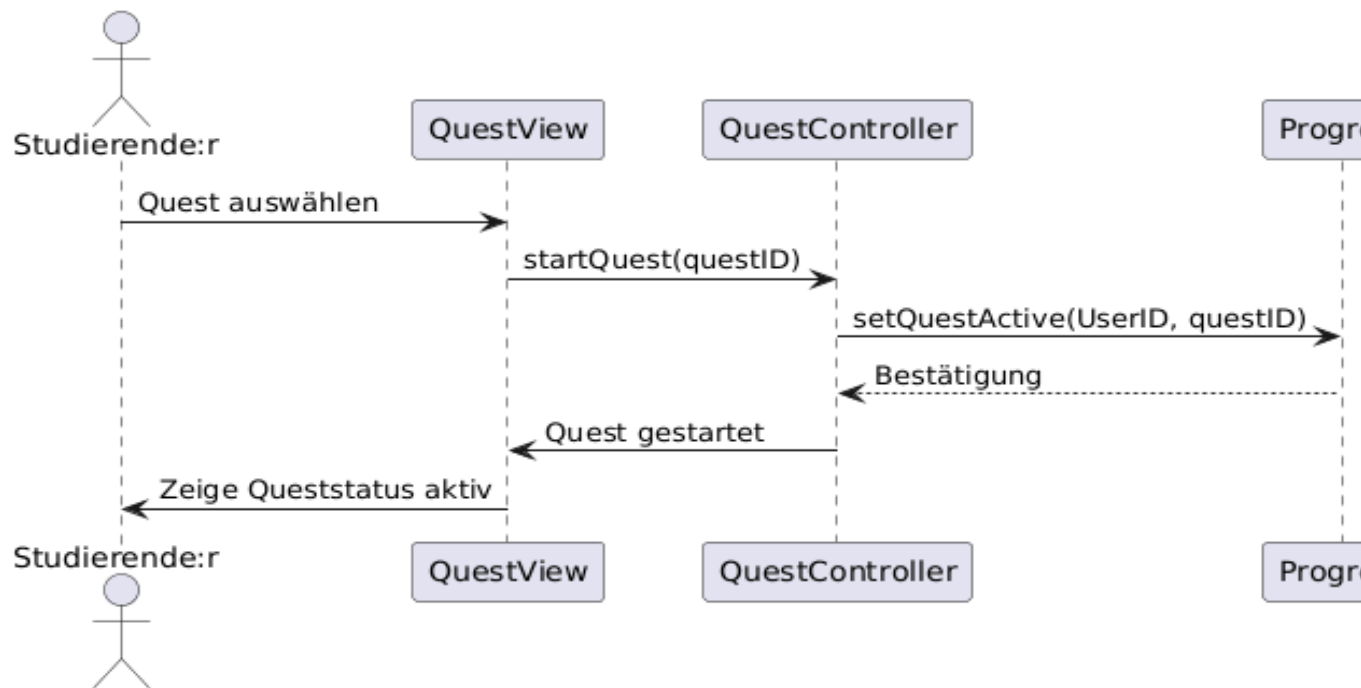
Beteiligte Klassen: User, Quest, UI (QuestView), DB

Referenzierte Requirements: UC04-F1, UC04-F2, UC04-F3, UC04-F4,

Hauptablauf:

1. Studierende:r öffnet die Quest-Übersicht (Authentifizierung geprüft, UC04-NF2).
2. UI lädt verfügbare Quests aus der DB und zeigt diese an (UC04-F1).
3. Studierende:r wählt eine Quest aus und klickt „Starten“.
4. System prüft, ob bereits eine aktive Quest existiert (UC04-F4).
5. Bei keiner aktiven Quest: User.setActiveQuest() wird aufgerufen, Startzeitpunkt gespeichert (UC04-F3).
6. Quest-Status wird auf „aktiv“ gesetzt (UC04-F2), Aktion protokolliert (UC04-NF3).
7. UI aktualisiert sich und zeigt die aktive Quest an (≤ 1 Sekunde, UC04-NF1).

Alternativpfad: - 4a: Bereits eine aktive Quest vorhanden → Fehlermeldung „Eine Quest ist bereits aktiv“ (UC04-F5). Start wird blockiert.



SQ05 – Lernquest abschließen / XP & Level-Up (UC05)

Akteure: Studierende:r

Beteiligte Klassen: User, Quest, LearningSession, GameRule, Achievement, Notification

Referenzierte Requirements: UC05-F1 bis UC05-F5, UC05-NF1 bis UC05-NF4

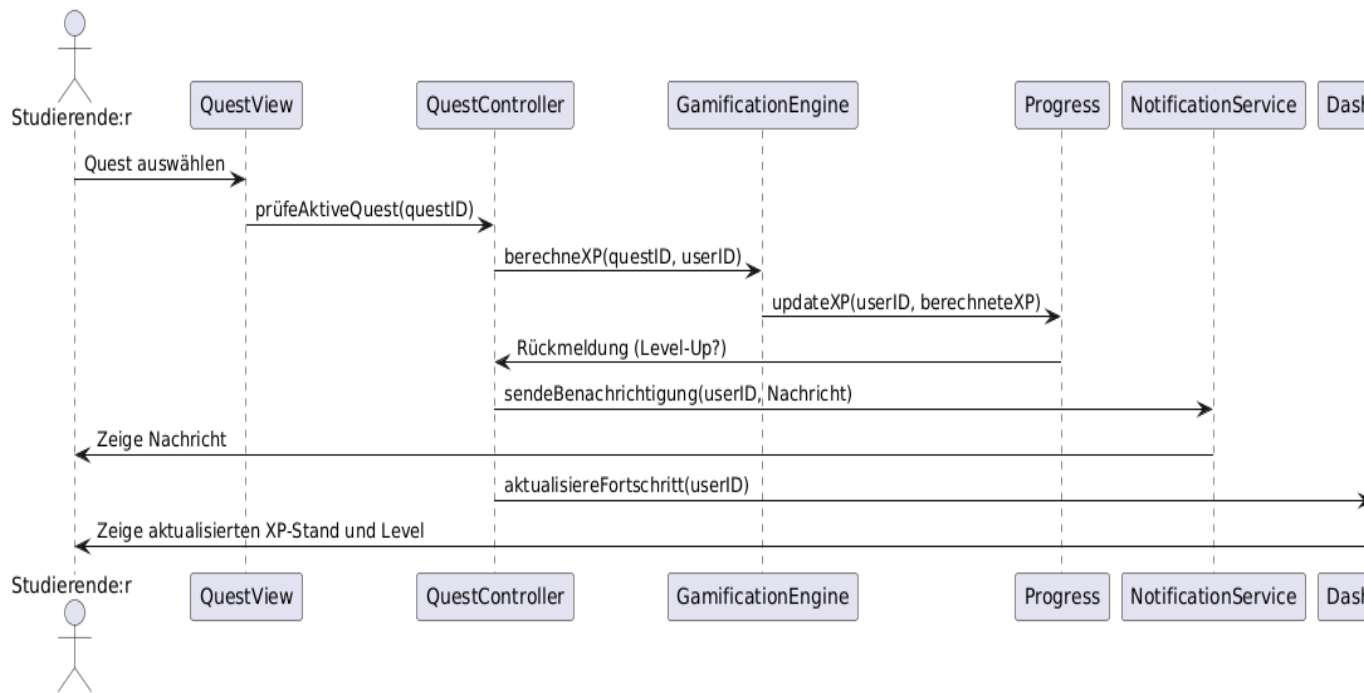
Hauptablauf:

1. Studierende:r klickt „Quest abschließen“ bei der aktiven Quest (UC05-F1).
2. System führt eine **atomare Transaktion** aus (UC05-NF2 – Doppel-Submit-Schutz):
 - a. GameRule.getRules() liefert aktuelle XP-Werte.
 - b. Quest.calculateXP() berechnet XP basierend auf Schwierigkeit (UC05-F2).
 - c. LearningSession.applyTimerBonus() berechnet optionalen

Timer-Bonus. d. User.addXP() addiert Gesamt-XP, prüft Level-Up (UC05-F3). e. LearningSession.logSession() speichert Session mit Endzeitpunkt (UC05-F5). f. Quest-Status wird auf „abgeschlossen“ gesetzt.

3. Achievement.checkUnlock() prüft, ob neue Badges freigeschaltet werden.
4. Bei Level-Up: Notification erstellt Level-Up-Benachrichtigung, UI zeigt visuelles Feedback (UC05-F4).
5. UI aktualisiert Dashboard (≤ 2 Sekunden Reaktionszeit, UC05-NF1).
6. Event wird protokolliert: „QUEST_COMPLETED“ (UC05-NF3).

Alternativpfad: - 1a: Keine aktive Quest → Button deaktiviert (UC05-NF4).



SQ06 – Lern-Session per Timer tracken (UC06)

Akteure: Studierende:r

Beteiligte Klassen: User, LearningSession, Quest, UI (TimerView)

Referenzierte Requirements: UC06-F1 bis UC06-F6, UC06-NF1 bis UC06-NF4

Hauptablauf:

1. Studierende:r klickt „Timer starten“ während einer aktiven Quest (UC06-F1).
2. System prüft, ob bereits ein aktiver Timer existiert (UC06-F6).
3. LearningSession wird erstellt, Startzeit gespeichert (UC06-F2).
4. UI zeigt laufenden Timer an (Reaktionszeit ≤ 1 Sekunde, UC06-NF4).

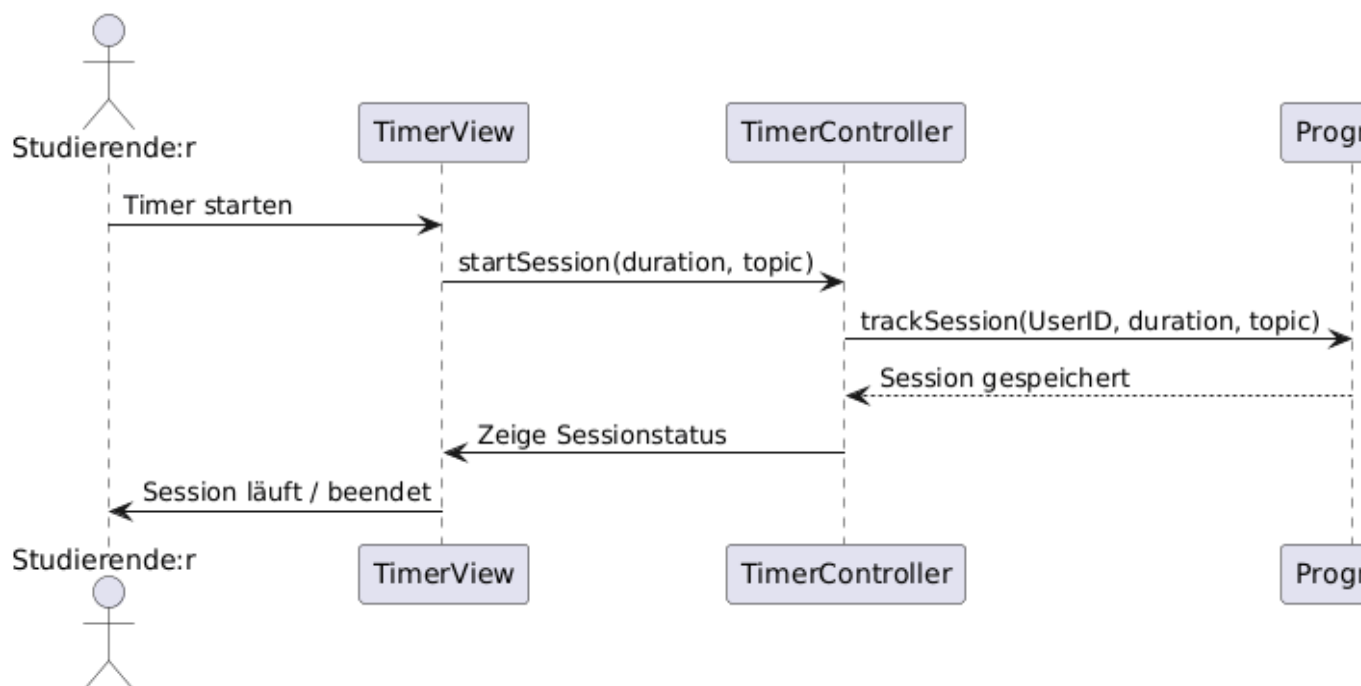
5. Studierende:r klickt „Timer stoppen“.

6. `LearningSession.calculateDuration()` berechnet Gesamtdauer (UC06-F4).

7. System vergibt optional XP basierend auf Lernzeit (UC06-F5): $XP = \text{duration_minutes} \times \text{xp_per_minute_timer}$.

8. Alle Timer-Aktionen werden protokolliert (UC06-NF3).

Alternativpfad: - **2a:** Timer bereits aktiv → Fehlermeldung „Nur ein aktiver Timer erlaubt“ (UC06-F6). - **4a:** Browser-Reload → Timerdaten bleiben erhalten (UC06-NF2).



SQ07 – Noten & Module verwalten (UC07)

Akteure: Studierende:r

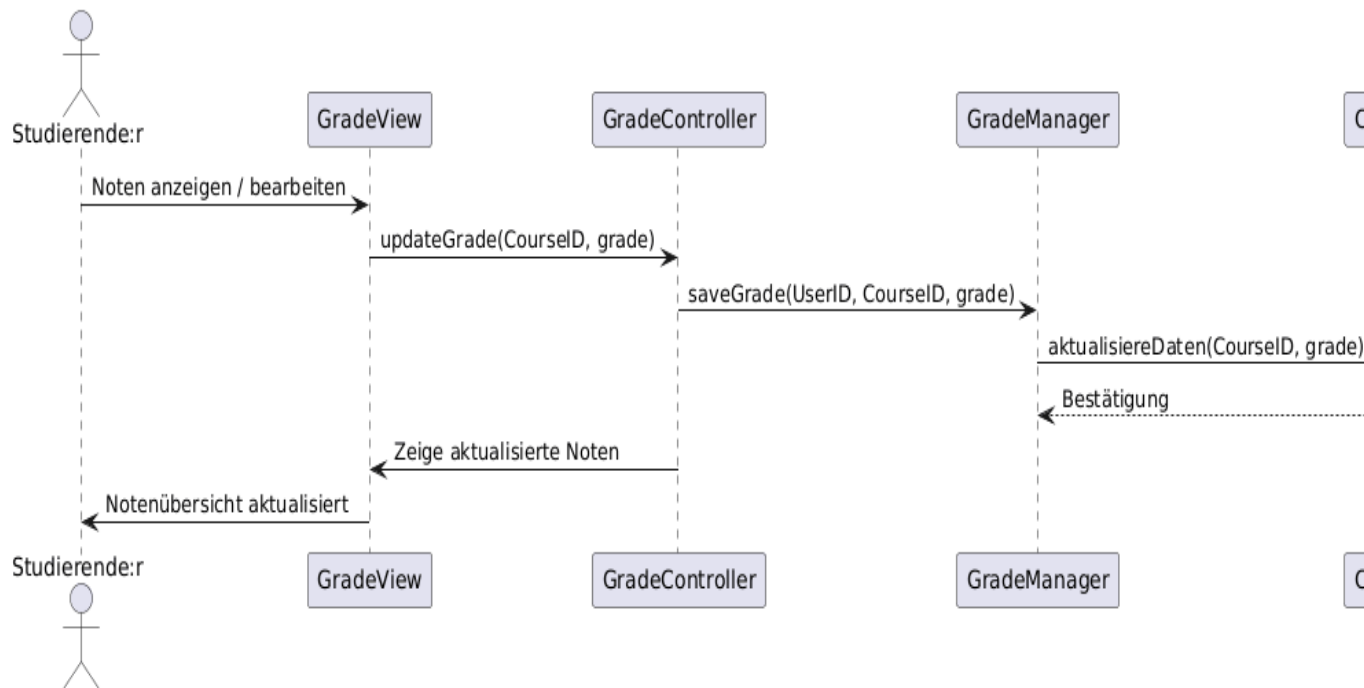
Beteiligte Klassen: User, Grade, UI (GradeView), DB

Referenzierte Requirements: UC07-F1 bis UC07-F5, UC07-NF1, UC07-NF2

Hauptablauf:

1. Studierende:r navigiert zur Notenübersicht.
2. UI lädt alle gespeicherten Noten aus der DB und zeigt sie an.
3. Studierende:r kann:
 - **Neue Note eintragen** (UC07-F1): Modul, Note und Gewichtung eingeben → Grade wird erstellt und gespeichert.
 - **Note bearbeiten** (UC07-F2): Bestehende Note ändern → Grade wird aktualisiert.
 - **Note löschen** (UC07-F3): Note entfernen → Grade wird aus DB gelöscht.
4. Nach jeder Änderung wird die Notenübersicht automatisch aktualisiert (UC07-F5, UC07-NF1 – ohne Seiten-Neuladen).
5. Datenkonsistenz wird auch bei schnellen aufeinanderfolgenden Operationen sichergestellt (UC07-NF2).

Alternativpfad: - 3a: Ungültige Notenwerte (außerhalb 1.0–6.0) → Validierungsfehler.



SQ08 – Fortschritt & Dashboard einsehen (UC08)

Akteure: Studierende:r

Beteiligte Klassen: User, Quest, Grade, Achievement, Leaderboard, UI (DashboardView)

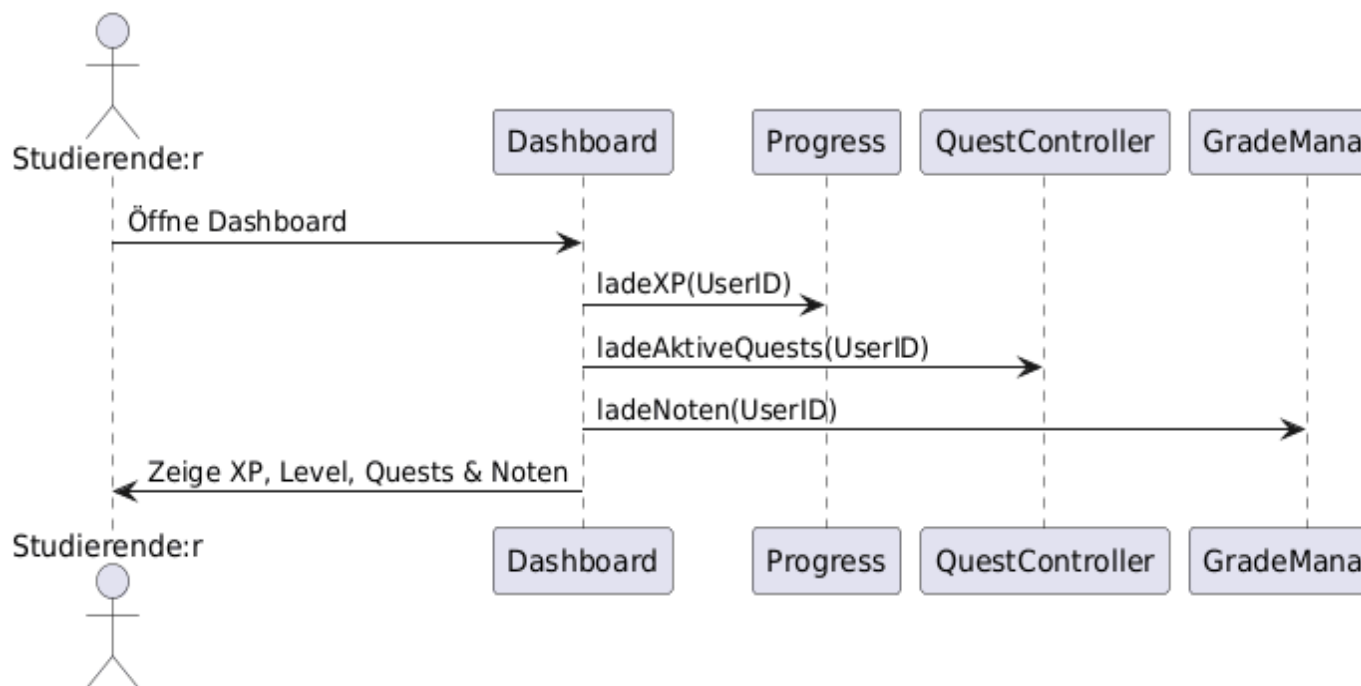
Referenzierte Requirements: UC08-F1 bis UC08-F5, UC08-NF1, UC08-NF2

Hauptablauf:

1. Studierende:r klickt auf „Dashboard“ oder wird nach Login automatisch weitergeleitet (UC08-F1).
2. UI aggregiert Daten aus mehreren Klassen parallel: - User: Aktueller XP-Stand (UC08-F2), Level (UC08-F3). - Quest: Aktive und abgeschlossene Quests (UC08-F4). - Grade: Notenstatistik und Durchschnitt (UC08-F5). - Achievement: Freigeschaltete Badges. - Leaderboard: Aktuelle Rangposition.

3. Dashboard wird vollständig geladen und angezeigt (≤ 2 Sekunden, UC08-NF1).
4. Darstellung ist browserübergreifend konsistent (UC08-NF2).

Alternativpfad: - 2a: Keine Daten vorhanden (neuer Benutzer) → Dashboard zeigt Standardwerte (Level 1, 0 XP, keine Quests).



SQ09 – Benachrichtigungen & Reminder verwalten (UC09)

Akteure: Studierende:r

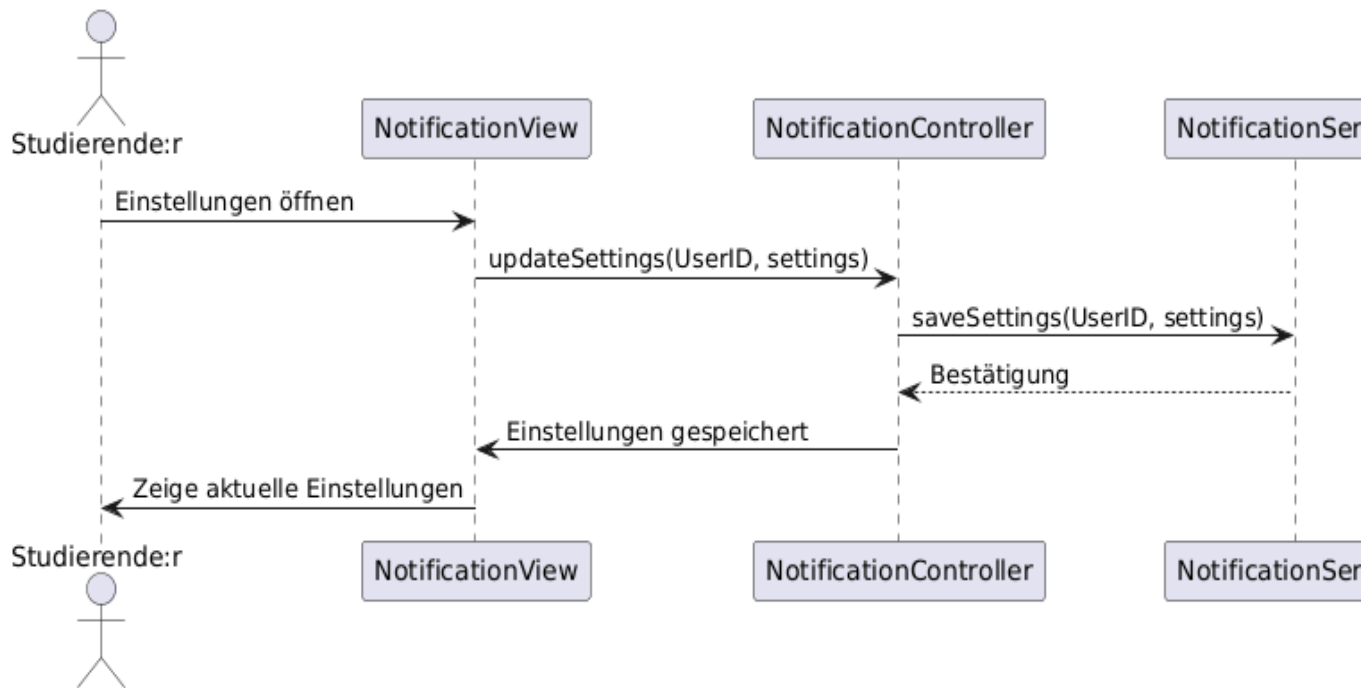
Beteiligte Klassen: User, Notification, UI (NotificationView)

Referenzierte Requirements: UC09-F1 bis UC09-F4, UC09-NF1, UC09-NF2

Hauptablauf:

1. System erstellt automatisch Benachrichtigungen bei Ereignissen (Quest-Abschluss, Level-Up, Achievement).
2. `Notification.createNotification()` speichert Nachricht mit Typ, Text und Zeitstempel.
3. UI zeigt ungelesene Benachrichtigungen als Badge/Glocken-Icon an.
4. Studierende:r öffnet Benachrichtigungsübersicht (UC09-F1).
5. Studierende:r kann Benachrichtigungen als gelesen markieren → `Notification.markAsRead()` (UC09-F4).
6. Änderungen wirken sofort ohne Neu-Laden (UC09-NF1).

Alternativpfad: - 4a: Keine ungelesenen Benachrichtigungen → Leere Liste mit Hinweistext.



SQ10 – Noten importieren / exportieren (UC10)

Akteure: Studierende:r

Beteiligte Klassen: User, Grade, UI (GradeView)

Referenzierte Requirements: UC10-F1, UC10-F2, UC10-F3, UC10-NF1

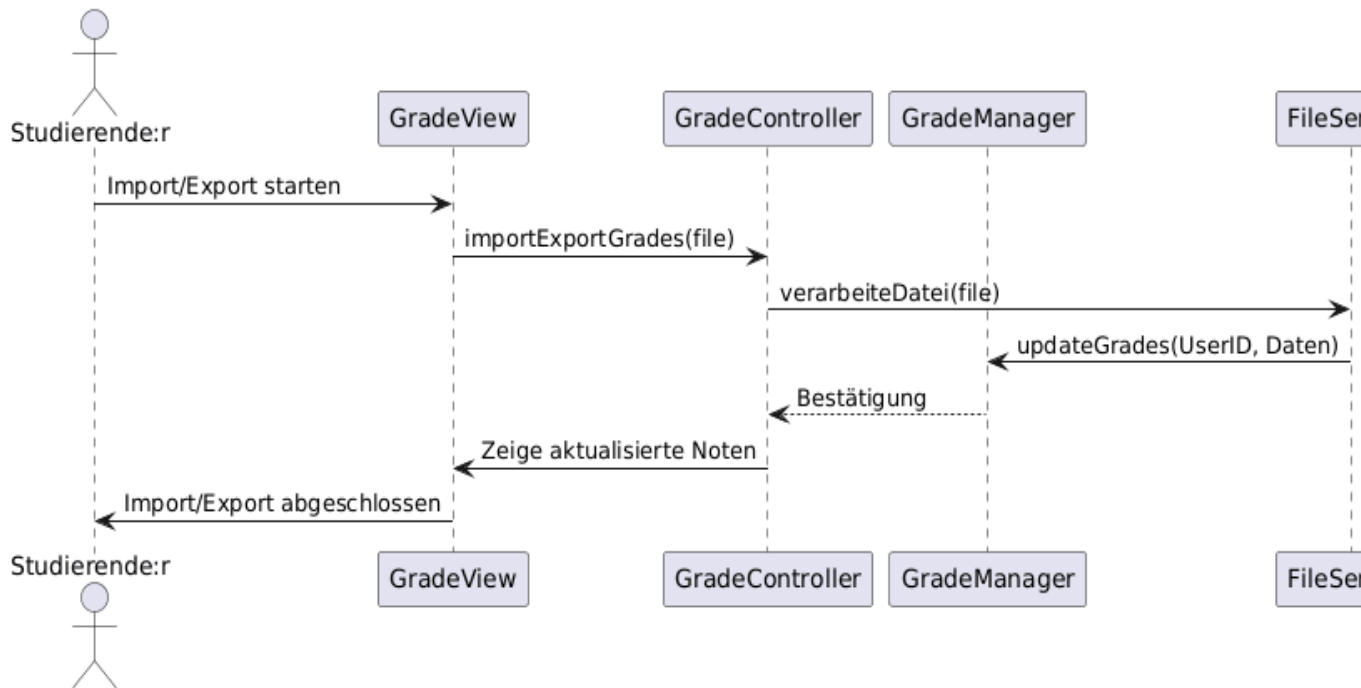
Szenario A: CSV-Import

Hauptablauf: 1. Studierende:r klickt „Noten importieren“ und wählt eine CSV-Datei. 2. `Grade.importFromCSV()` parst die CSV-Datei und validiert die Daten. 3. Bei gültigen Daten: Noten werden in die DB importiert. 4. UI aktualisiert die Notenübersicht mit den neuen Einträgen.

Alternativpfad: - 2a: Ungültige CSV-Datei (falsches Format, fehlende Spalten)
→ Fehlermeldung (UC10-F3).

Szenario B: CSV-Export

Hauptablauf: 1. Studierende:r klickt „Noten exportieren“. 2. `Grade.exportToCSV()` generiert eine CSV-Datei aus allen gespeicherten Noten. 3. Datei wird zum Download angeboten. 4. Gesamtvorgang dauert ≤ 5 Sekunden (UC10-NF1).



SQ11 – Achievements / Badges freischalten (UC11)

Akteure: Studierende:r (passiv), System (aktiv)

Beteiligte Klassen: User, Achievement, Notification

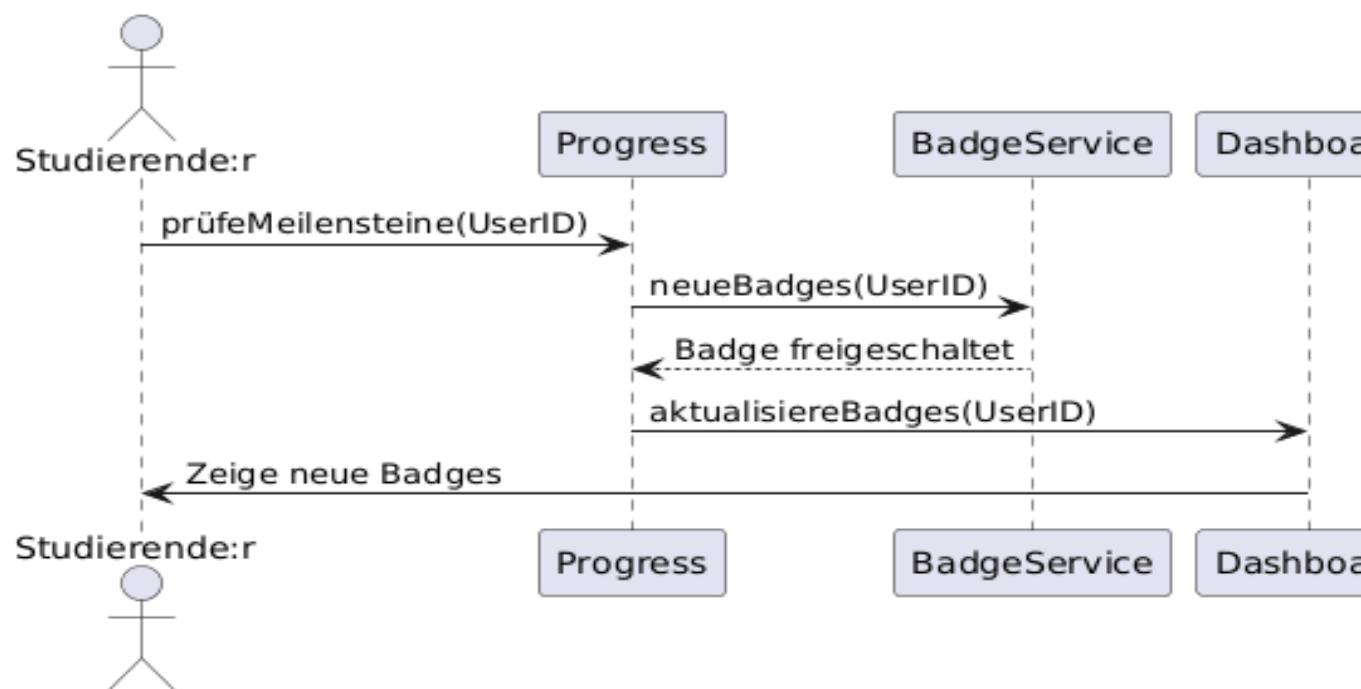
Referenzierte Requirements: UC11-F1 bis UC11-F4, UC11-NF1

Hauptablauf:

1. Nach jeder relevanten Aktion (Quest-Abschluss, Timer-Session, Streak-Tag) wird automatisch Achievement.checkUnlock() getriggert (UC11-NF1).
2. System prüft alle Achievement-Bedingungen gegen den aktuellen User-Stand (UC11-F1).
3. Bei erfüllter Bedingung: Achievement.unlockForUser() schaltet das Badge frei (UC11-F2).

4. Notification wird erstellt und UI zeigt visuelles Feedback (Popup/Animation, UC11-F3).
5. Studierende:r kann alle Badges (freigeschaltet + gesperrt) in der Übersicht einsehen (UC11-F4).

Alternativpfad: - 2a: Keine Bedingung erfüllt → Kein Badge freigeschaltet, kein Feedback.



SQ12 – Leaderboard & Streaks anzeigen (UC12)

Akteure: Studierende:r

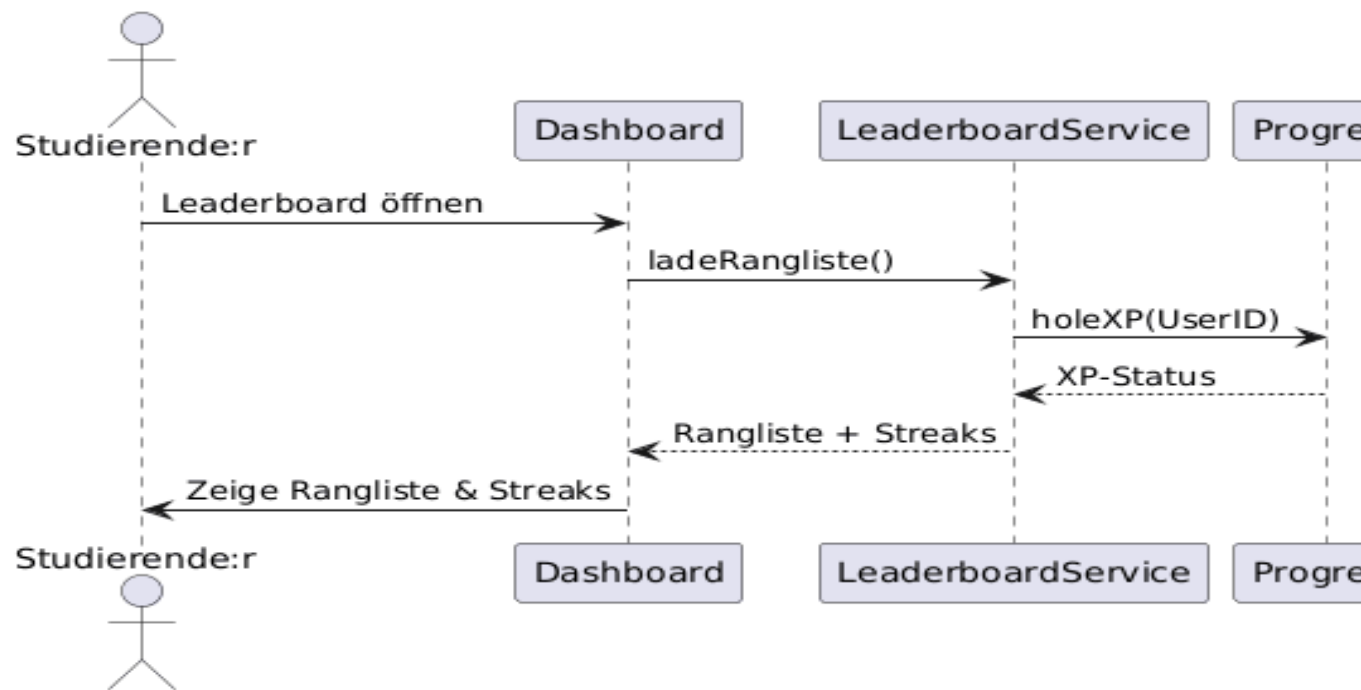
Beteiligte Klassen: User, Leaderboard, UI (LeaderboardView)

Referenzierte Requirements: UC12-F1 bis UC12-F4, UC12-NF1, UC12-NF2

Hauptablauf:

1. Studierende:r navigiert zum Leaderboard.
2. UI ruft `Leaderboard.calculateRankings()` auf – Rankings werden nach gewähltem Kriterium berechnet (UC12-F1): XP, Level, Quests oder Streak.
3. `Leaderboard.updateStreaks()` berechnet aktuelle Streaks aller User (UC12-F2).
4. UI zeigt die Rangliste an, eigene Position wird hervorgehoben (UC12-F4).
5. Daten werden bei jedem Aufruf aktualisiert (UC12-F3).
6. Ladezeit ≤ 3 Sekunden (UC12-NF1).

Alternativpfad: - 2a: Nur ein Benutzer im System → Leaderboard zeigt Einzeleintrag.



SQ13 – Quest-Katalog verwalten (UC13, Admin)

Akteure: Administrator

Beteiligte Klassen: User (Admin), Quest, GameRule, UI (AdminView), DB

Referenzierte Requirements: UC13-F1 bis UC13-F6, UC13-NF1, UC13-NF2

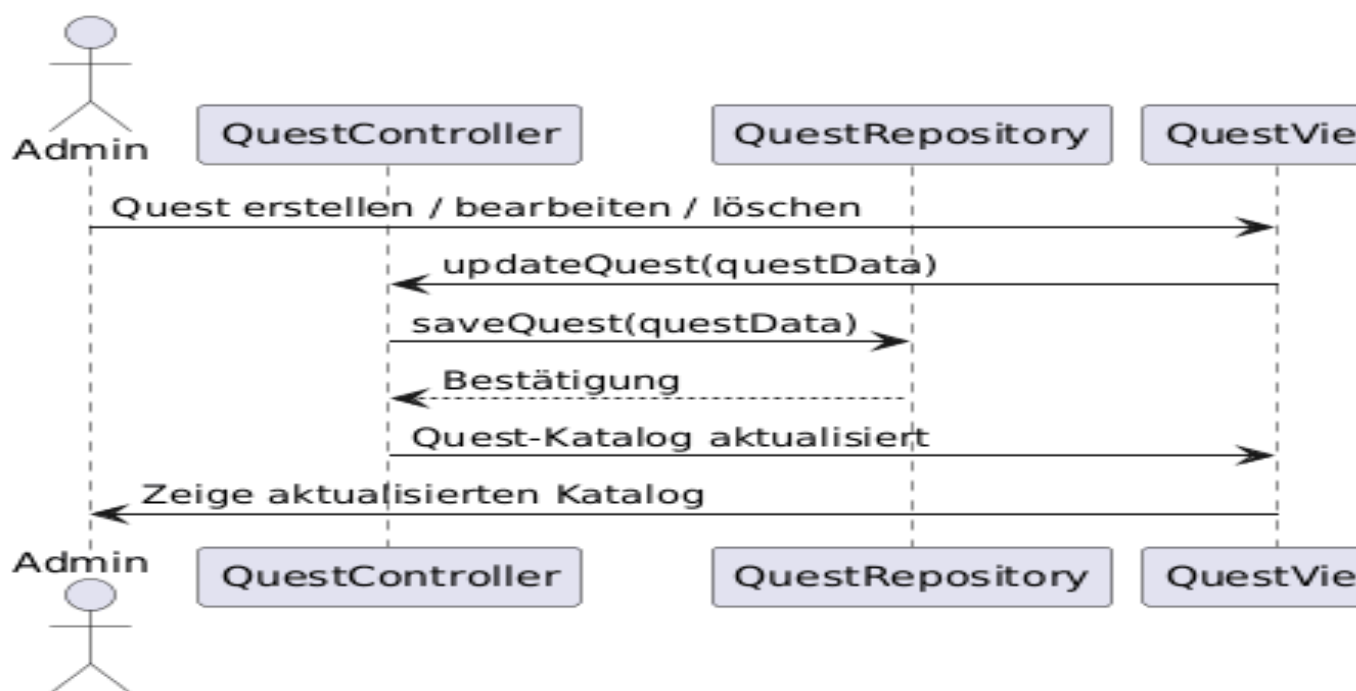
Hauptablauf:

1. Administrator meldet sich an (UC13-F1, Rollenprüfung UC13-NF2).
2. Admin navigiert zu „Quests verwalten“ (UC13-F2).
3. UI zeigt den Quest-Katalog mit allen vorhandenen Quests.
4. Admin kann: - **Quest erstellen** (UC13-F3): Titel, Beschreibung, Schwierigkeit eingeben → Quest wird in DB gespeichert. - **Quest**

bearbeiten (UC13-F4): Bestehende Quest-Daten ändern → Update in DB. -
Quest löschen (UC13-F5): Quest aus dem Katalog entfernen.

5. Katalog aktualisiert sich nach jeder Änderung (UC13-F6, ≤ 2 Sekunden, UC13-NF1).

Alternativpfad: - 1a: Nicht-Admin-Benutzer → Zugriff verweigert, Redirect zum Dashboard.



SQ14 – XP- und Levelregeln konfigurieren (UC14, Admin)

Akteure: Administrator

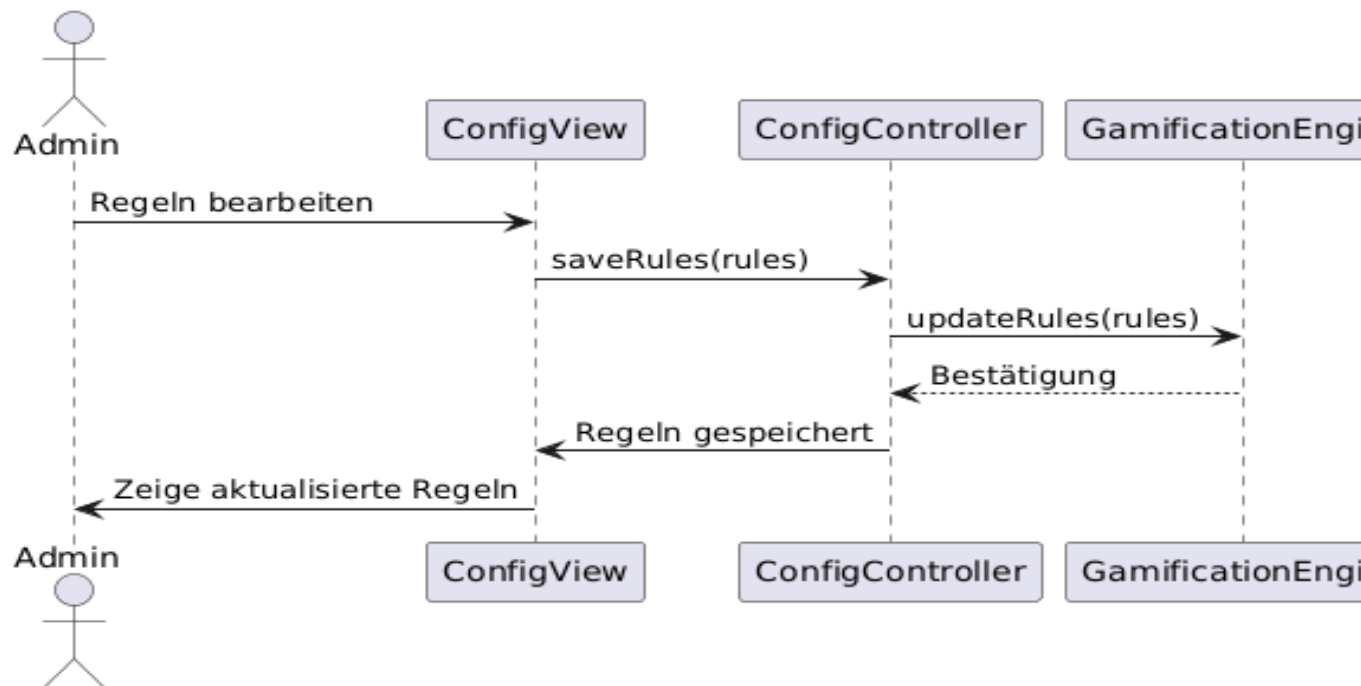
Beteiligte Klassen: User (Admin), GameRule, UI (AdminView), DB

Referenzierte Requirements: UC14-F1 bis UC14-F5, UC14-NF1, UC14-NF3, UC14-NF4

Hauptablauf:

1. Administrator authentifiziert sich (UC14-F1, Rollenprüfung UC14-NF4).
2. Admin navigiert zu „Regeln verwalten“ (UC14-F2).
3. UI zeigt aktuelle GameRule-Werte (easy_xp, medium_xp, hard_xp, level_threshold, xp_per_minute_timer, max_level).
4. Admin ändert gewünschte Werte (UC14-F3).
5. GameRule.updateRules() speichert die neuen Werte (UC14-F4, Speicherdauer ≤ 2 Sekunden, UC14-NF3).
6. Neue Regeln gelten sofort global für alle Benutzer (UC14-F5).
7. System stellt sicher, dass keine Dateninkonsistenzen entstehen (UC14-NF1).

Alternativpfad: - 4a: Ungültige Werte (z.B. negative XP) → Validierungsfehler, Werte werden nicht gespeichert.



SQ15 – Benutzerkonten administrieren (UC15, Admin)

Akteure: Administrator

Beteiligte Klassen: User (Admin), User (Ziel-Benutzer), UI (AdminView), DB

Referenzierte Requirements: UC15-F1 bis UC15-F5, UC15-NF1, UC15-NF3

Hauptablauf:

1. Administrator meldet sich an (UC15-F1, Rollenprüfung UC15-NF1).
2. Admin sucht nach Benutzerkonten (UC15-F2) über Name oder E-Mail.
3. UI zeigt Treffer mit Kontodetails an (UC15-F3).

4. Admin wählt eine Aktion (UC15-F4): - **Sperren**: User.is_active = false → Benutzer kann sich nicht mehr einloggen. - **Reaktivieren**: User.is_active = true → Zugang wiederhergestellt. - **Löschen**: Benutzerkonto wird aus der DB entfernt.
5. System aktualisiert den Kontostatus (UC15-F5, Reaktionszeit ≤ 2 Sekunden, UC15-NF3).

Alternativpfad: - **2a:** Keine Treffer bei der Suche → Hinweis „Kein Benutzer gefunden“. - **4a:** Admin versucht eigenes Konto zu löschen → Aktion wird blockiert.

